

The search continues: IF2 workflow and example

Jesse Wheeler Qianying (Ruby) Lin Spencer J. Fox

Previously on... Simulation-Based Inference

What we have accomplished so far:

1. Built a continuous-time stochastic **SIR** model in `pomp`.
2. Simulated outbreaks to understand underlying dynamics.
3. Used the particle filter (`pfilter`) to estimate the likelihood of a specific parameter set given our data.
4. Used Iterated Filtering (`mif2`) to perform a **local search**, allowing the algorithm to climb the likelihood surface to a peak.

What is next:

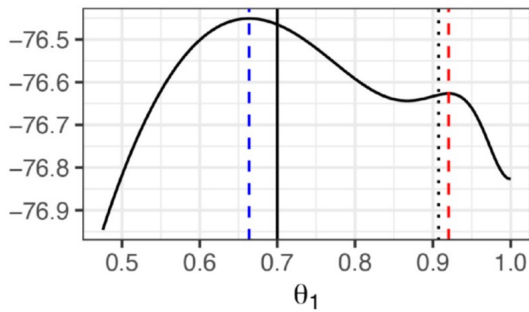
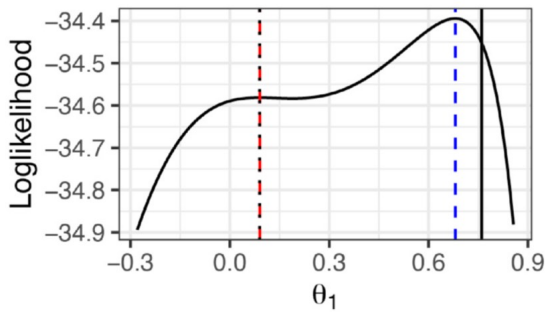
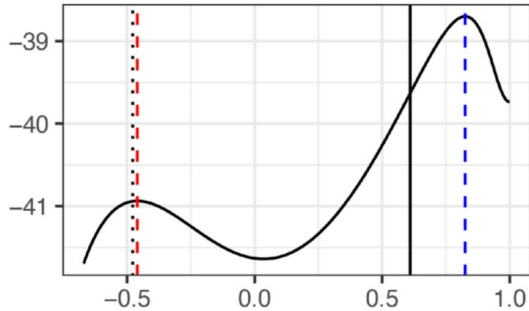
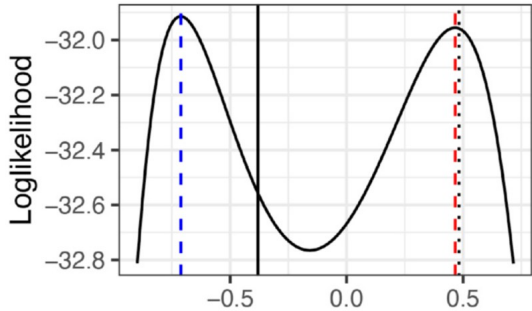
- ▶ **Global Search:** How do we know we found the *highest* peak (Global Maximum)?
- ▶ **Uncertainty:** How do we calculate Confidence Intervals (CIs) for our parameter estimates?

The Problem with Local Search

- ▶ `mif2` is a powerful optimization algorithm, but is a **local** optimizer with finite computational effort.
- ▶ We have also seen that the output in this case is also random!
- ▶ **Remember:** an MLE (almost) always exists, as likelihood is bounded. To perform inference, we need to reliably maximize the likelihood, up to tolerable Monte Carlo Error.

Other dangers:

- ▶ In complex epidemiological models (like SIR), the likelihood surface is often rugged, with many local maxima, ridges, and flat valleys.
- ▶ **The Danger:** If you only start from one “best guess”, you might get stuck on a small hill while the true global maximum is mountains away.



How do you know when you've found it?

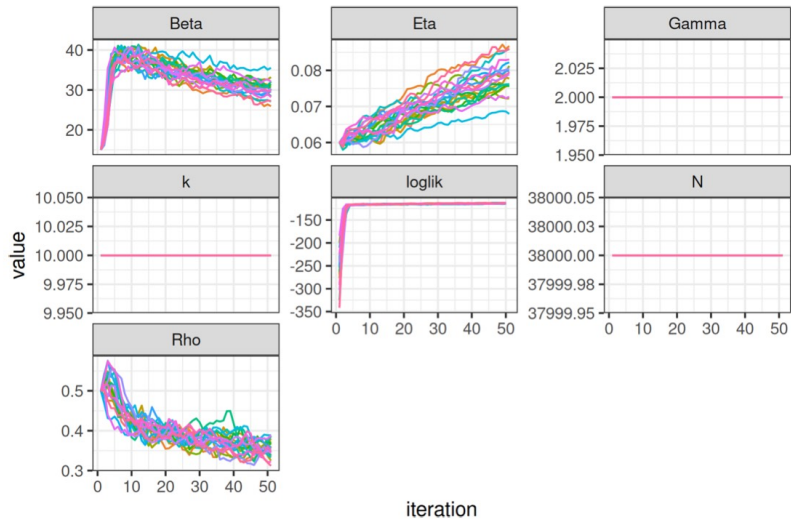
- ▶ Trace Plots (has likelihood stabilized)?
- ▶ Performing global search for parameter estimates.
- ▶ Comparing log-likelihood of Monte Carlo replicates.
- ▶ Computing confidence intervals.

Trace plots diagnostics

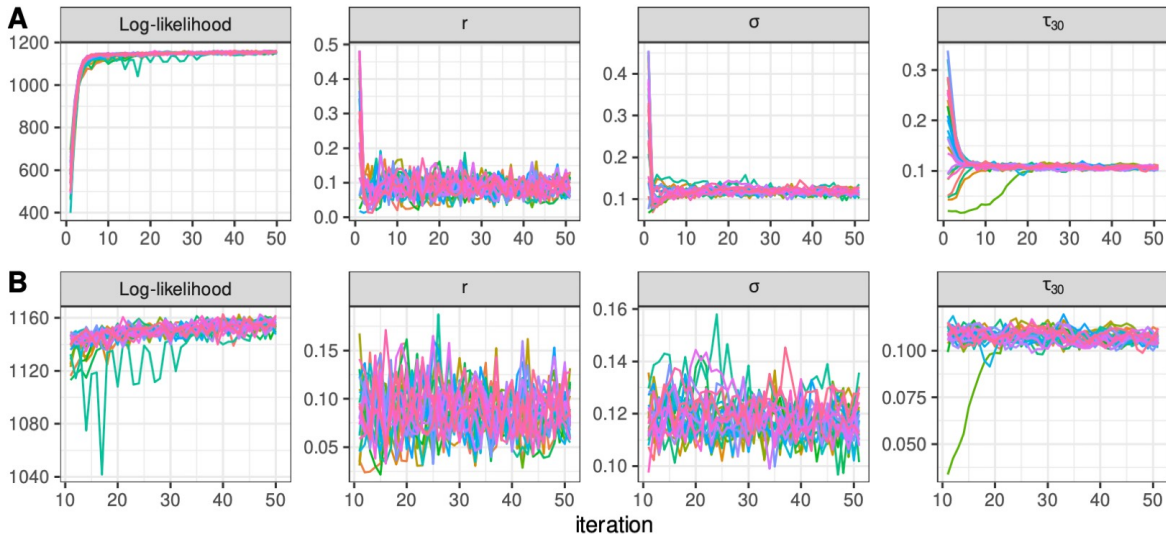
Along with effective sample size, one key diagnostic is the “trace-plot”.

- ▶ We can extract the entire history of each search using the `trace()` function in `pomp`, and plot changes in likelihood and parameter values.
- ▶ We want to see convergence to a region of high-likelihood, and for the search to “stabilize”.

Trace plot: Not finished



Trace plot: Zoomed in result



Tace plot: Code

```
foreach(i=1:20,.combine=c) %dopar% {  
  measSIR |>  
    mif2(  
      Np=2000, Nmif=50, cooling.fraction.50=0.5,  
      rw.sd=rw_sd(Beta=0.02, Rho=0.02, Eta=ivp(0.02))  
    )  
} -> mifs_local  
  
mifs_local |>  
  traces() |>  
  melt() |>  
  ggplot(aes(x=iteration,y=value,group=.L1,color=factor(.L1)))+  
  geom_line()+  
  guides(color="none")+  
  facet_wrap(~name,scales="free_y")
```

Global Search: idea

Another way to ensure we have obtained the MLE is to more thoroughly search all regions of the likelihood surface.

Global Search: practice I

To conduct a global search, we cast a wide net across the parameter space.

1. Define a large “box” (hypercube) of biologically plausible parameter values.
2. Draw a large sample of random starting points from within this box.
3. Run a local search (`mif2`) starting from *each* of those points.
4. Compare the final likelihoods of all runs. The highest peak found is our Maximum Likelihood Estimate (MLE).

Global Search: practice II

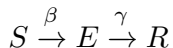
We will build on the previous example / code:

```
source("scripts/model_measSIR.R")

# Transformations to stay "in-bounds":
#   - log: parameters must be positive.
#   - logit: parameters in (0, 1)
measSIR <- measSIR |> pomp(
  partrans = parameter_trans(
    log=c("Beta", "Gamma", "k"),logit=c("Rho","Eta")
  ),
  paramnames = c("Beta","Gamma","Eta","Rho","k","N")
)
```

Model reminder

Reminder: The Baseline SIR Model Before we search the parameter space, let's recall the exact continuous-time SIR model we are fitting:



Continuous-Time Jump Processes: We simulate this using Euler-Binomial approximations over time step dt :

$$N_{SI}(t) \sim \text{Binom} \left(S(t), 1 - \exp \left(-\beta \frac{I(t)}{N} dt \right) \right)$$

$$N_{IR}(t) \sim \text{Binom} (I(t), 1 - \exp(-\gamma dt))$$

ρ is reporting rate, k is dispersion parameter, and η is the initial susceptible proportion.

Defining fixed parameters

We can always choose to fix or estimate parameters. For demonstration purposes, we will fix some and estimate others.

- ▶ Fix $\gamma = 2$ / week.
- ▶ Fix $k = 10$.
- ▶ Also fix population size $N = 38000$, because it is known.

```
fixed_params <- c(N=38000, Gamma=2, k=10)
```

Defining the Parameter Box for estimated params

First, we set plausible upper and lower bounds for our unknown SIR parameters.

```
set.seed(2062379496)

global_starts <- runif_design(
  lower=c(Beta=5,Rho=0.2,Eta=0),
  upper=c(Beta=80,Rho=0.9,Eta=1),
  nseq=400
)
```

Parameter box (continued)

```
head(global_starts)
```

	Beta	Rho	Eta
	28.416342	0.6851642	0.7684210
	49.838633	0.4615048	0.3889003
	32.219926	0.6659006	0.1403907
	37.342160	0.4401953	0.3487979
	5.791888	0.6922474	0.7062618
	25.610028	0.7985045	0.8998132

Running the Global Search (Parallelized)

Because we are running many independent mif2 searches, this is perfectly suited for parallel computing using foreach and %dopar%.

```
library(foreach)
library(doParallel)
library(doRNG)
library(iterators)

registerDoParallel()
registerDoRNG(123456)
```

Running the search

```
global_mifs <- foreach(  
  guess = iter(global_starts, "row"), .combine = c, .packages = c("pomp")  
  
  # Combine fixed and random initializations  
  start = unlist(c(fixed, guess))  
  
  measSIR |> mif2(  
    Np=2000, Nmif=100, cooling.fraction.50=0.5,  
    params = start, # use this as starting location  
    rw.sd=rw_sd(Beta=0.02, Rho=0.02, Eta=ivp(0.02))  
  )  
}
```

Note: Long-search times

- ▶ If you pick many starting values, and large number of iterations and particles, a global search can take a long time.
- ▶ We can estimate how long it will take, and use that to determine how much effort is reasonable on a laptop before moving to a HCP

```
system.time(pfilter(measSIR, Np = 1000))
```

- ▶ Computing time scales linearly in J , M , and the number of replicates.
- ▶ Divide by number of cores.

Example calculation

Here's a basic example. Suppose that we did a small local search with $M = 50$, $J = 2000$ replicated 16 times, using 10 cores, and that it took 3 minutes. From this, we can conclude each individual search takes approximately $3 \text{ min} / \text{ceiling}(16/10) = 1.5 \text{ min}$.

Then, if I want to use $M = 75$ iterations, $J = 2500$ particles, with 87 replicates on the same computer, then the total computing time would be approximately:

$$1.5 \text{ min} \times (75/50) \times (2500/2000) \times \text{ceiling}(87/10) \approx 25.3 \text{ min}$$

Evaluating the Global Search

After the searches finish, we extract the likelihoods (using `pfilter`) and parameter estimates, usually combining them into a data frame.

```
# Evaluate likelihoods for the final state of each mif2 run
global_lik <- foreach(m = global_mifs, .combine = rbind) %dopar% {
  evals <- replicate(10, pfilter(m, Np = 5000) |> logLik())
  ll <- logmeanexp(evals, se = TRUE)
  m |> coef() |> bind_rows() |> bind_cols(loglik=ll[1],loglik.se=ll[2])
} %>% as.data.frame()
```

Keeping all search results

Because we're actively exploring the likelihood surface, searching for the MLE and ridges, keep all previous results!

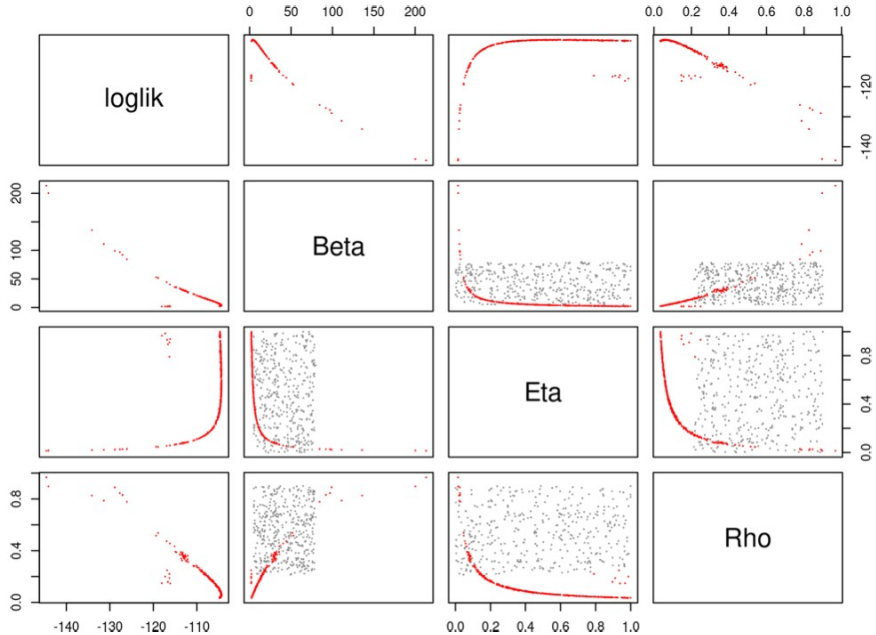
```
read_csv("measles_params.csv") |>  
  bind_rows(global_lik) |>  
  arrange(-loglik) |>  
  write_csv("measles_params.csv")
```

Exploring parameter surface

As we build our global search, we start to get a good idea of what the parameter surface looks like

```
read_csv("measles_params.csv") |>
  filter(loglik>max(loglik)-50) |>
  bind_rows(guesses) |>
  mutate(type=if_else(is.na(loglik),"guess","result")) |>
  arrange(type) -> all

pairs(~loglik+Beta+Eta+Rho, data=all, pch=16, cex=0.3,
  col=ifelse(all$type=="guess",grey(0.5),"red"))
```



Point Estimates Are Not Enough

- ▶ Our search will often show we can achieve maximum likelihood for various parameter combinations (sharp ridges).
- ▶ We can narrow our search for specific parameter values of interest via profile confidence intervals.
- ▶ These compute the uncertainty for a single parameter, accounting for the effect of others.

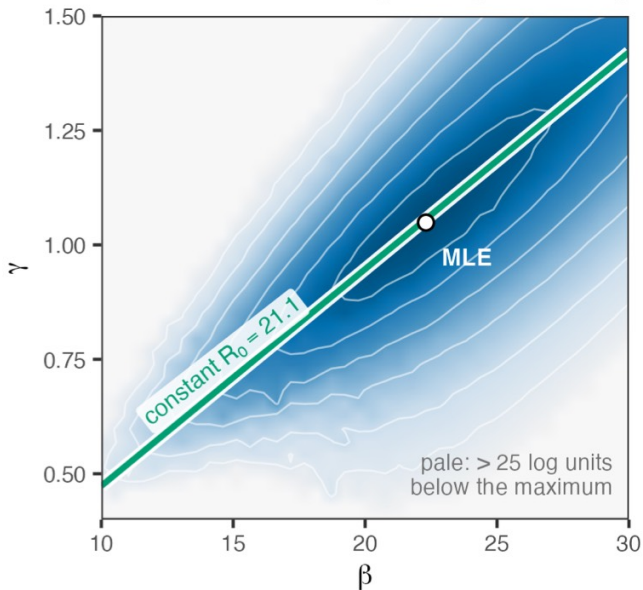
What is a Profile Likelihood?

Profile likelihood is used to integrate out the effect of other parameters. The **profile log-likelihood function** of ϕ is defined to be

$$\ell^{\text{profile}}(\phi) = \max_{\psi} \ell(\phi, \psi).$$

In general, the profile likelihood of one parameter is constructed by maximizing the likelihood function over all other parameters.

Darker = better fit. The good region is a ridge.



Computing a profile:

To find the confidence interval for a specific parameter (e.g., η):

1. We fix η at a specific value.
2. We maximize the likelihood over all other parameters.
3. We repeat this across a grid of η values.

The result: A curve showing the highest possible likelihood for the model at any given value of η .

Setting up the Profile Design

We use `profile_design()` to create a grid of fixed values for our parameter of interest (β), while generating random starting guesses for the parameters we still need to estimate.

```
# Create a grid for Beta (40 evenly spaced points)
eta_grid <- seq(0.01, 0.95, length=40)

# Generate profile design
profile_guesses <- profile_design(
  eta = eta_grid,
  lower = c(Beta = 5, Rho = 0.2),
  upper = c(Beta = 80, Rho = 0.9),
  nprof = 15 # 15 random starts per Beta value
)
```

Running the Profile Search

Just like the global search, we loop over these guesses in parallel. The key difference is that we must turn off the random walk for the focal parameter (β).

```
profile_mifs <- foreach(guess = iter(profile_guesses, "row"),
                        .combine = c,
                        .packages = c("pomp")) %dopar% {

  # MUST unlist() to convert the guess data.frame row into a named vector
  start_params <- unlist(c(fixed, guess))

  measSIR |> mif2(
    Np = 2000, Nmif = 100, cooling.fraction.50 = 0.5,
    params = start_params,
    # Turn off random walk for Eta so it stays fixed!
    rw.sd = rw_sd(Beta=0.02, Rho=0.02)
  )
}
```

Wilks' Theorem and the 95% CI Cutoff

How do we turn a profile likelihood curve into a Confidence Interval? We use Wilks' Theorem.

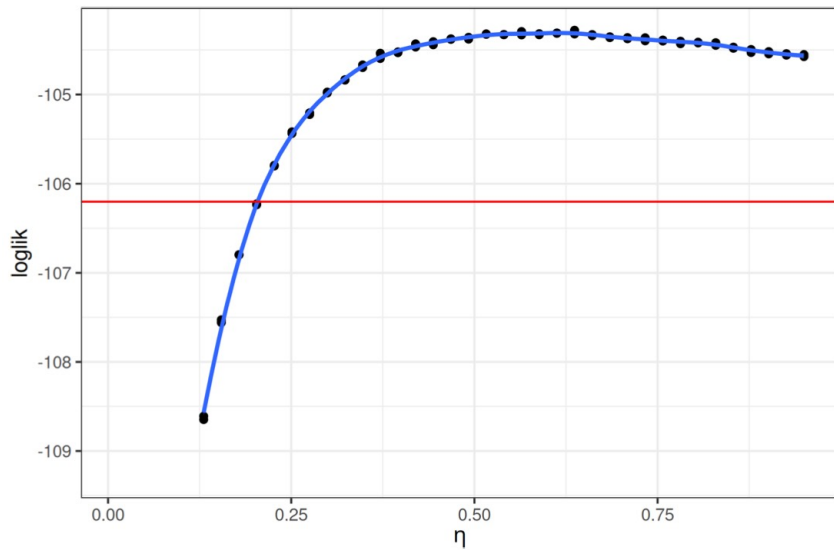
For a 95% CI on a single parameter, the cutoff is exactly 1.92 log-likelihood units below the global maximum likelihood.

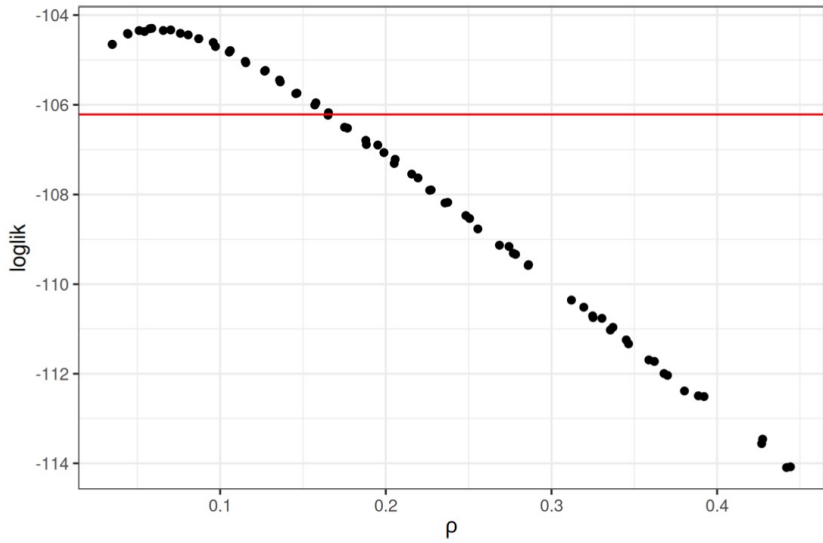
$$\text{Cutoff} = \max(\log\text{Lik}) - \frac{1}{2}\chi_{1,0.95}^2$$

$$\text{Cutoff} = \max(\log\text{Lik}) - 1.92$$

Any value of Beta that produces a log-likelihood above this cutoff is inside our 95% Confidence Interval!

Profile curves



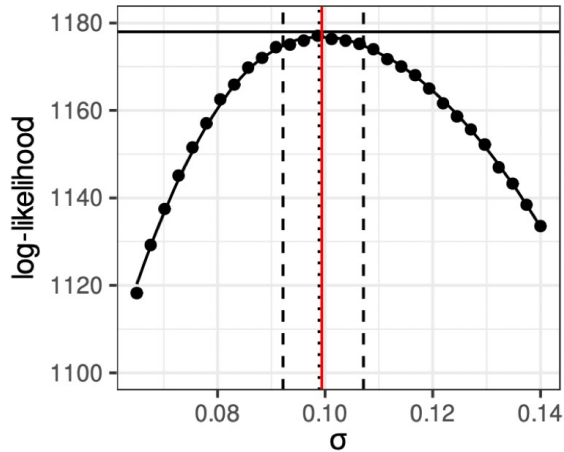
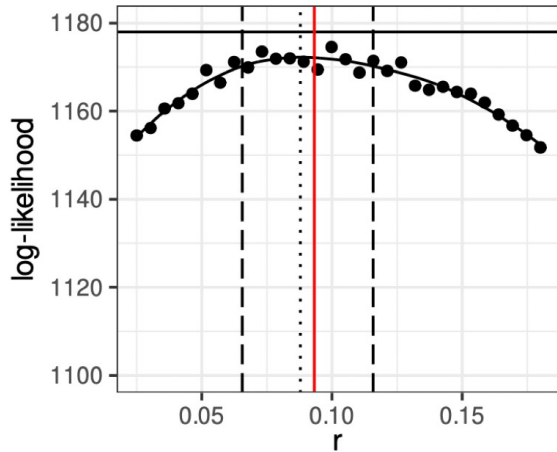


Accounting for parameter uncertainty

Like the standard log-likelihood function, we only have stochastic evaluations of the profile-likelihood.

- ▶ In our example, this variance is very low, and does not cause an issue.
- ▶ More often, we need to adjust for the Monte Carlo variance in profile estimates via Monte Carlo adjusted profile confidence intervals (Ionides et al. (2017)).

MCAP figure



MCAP Code Example I

```
prof_liks <- foreach(m = profile_mifs, .combine = rbind) %dopar% {  
  evals <- replicate(10, pfilter(m, Np = 5000) |> logLik())  
  ll <- logmeanexp(evals, se = TRUE)  
  m |> coef() |> bind_rows() |> bind_cols(loglik=ll[1],loglik.se=ll[2])  
} %>% as.data.frame()  
  
prof_liks |>  
  dplyr::group_by(Eta) |>  
  dplyr::slice_max(loglik)  
  
mcap(logLik = prof_liks$loglik, parameter = prof_liks$Eta)  
# Returns a list with a confidence interval, and smoothed profile.
```

Summary of the Inference Workflow

1. **Build & Simulate:** C-snippets, pomp object, check dynamics.
2. **Local Search:** Run `mif2` once to check particle filtering and optimization mechanics.
3. **Global Search:** Run `mif2` hundreds of times from a parameter hypercube to find the Global MLE.
4. **Profile Likelihood:** Fix a parameter across a grid, re-maximize the others, and apply Wilks' theorem to extract 95% Confidence Intervals.

Recap: Euler-Binomial Scheme

We previously considered continuous time models with a simple, one-option flow:

$$S \rightarrow E \rightarrow I \rightarrow R$$

We did this by turning continuous transition rates μ_{SE} into transition probabilities over a small time step dt :

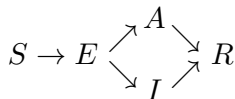
$$N_{SE}(t) \sim \text{Binom}(S(t), 1 - e^{-\mu_{SE} \times dt})$$

```
double p = 1 - exp(-mu_SE * dt);  
double trans_SE = rbinom(S, p); // Draw from Susceptible pool  
S -= trans_SE;  
E += trans_SE;
```

This works perfectly when there is only **one** way out of a compartment.

The Problem: Competing Risks

What happens when an individual can exit a compartment in multiple ways? Consider a model where an exposed individual may become either **symptomatic** (I), or **asymptomatic** (A):



If we use sequential `rbinom` draws, **the first draw always gets priority**, creating a mathematical bias:

```
double trans_I = rbinom(E, p_I);  
// Bias! Asymptomatic draw is artificially restricted by the first draw  
double trans_A = rbinom(E - trans_I, p_A);
```

Euler-Multinomial Distribution

To avoid this bias, we need to draw both transitions at the same time. We can handle convert rates into probabilities, and do simultaneous draws, with a single distribution:
Euler-Multinomial.

This is a natural extension of the process of converting continuous-time rates to probabilities, and doing binomial draws, which has two options: staying in E , or going to I .

Now, we may have 3 options: Staying in E , going to I , or going to A :

$$(N_{EI}(t), N_{EA}(t)) \sim \text{EulerMultinom}[E(t), (\mu_{EI}(t), \mu_{EA}(t))]$$

reulermultinom in practice

The `reulermultinom` function does this for us. It calculates the total exits based on the sum of all rates, then correctly distributes them.

```
// 1. Arrays for rates and output
double rates[2] = {mu_EI, mu_EA};
double trans[2];

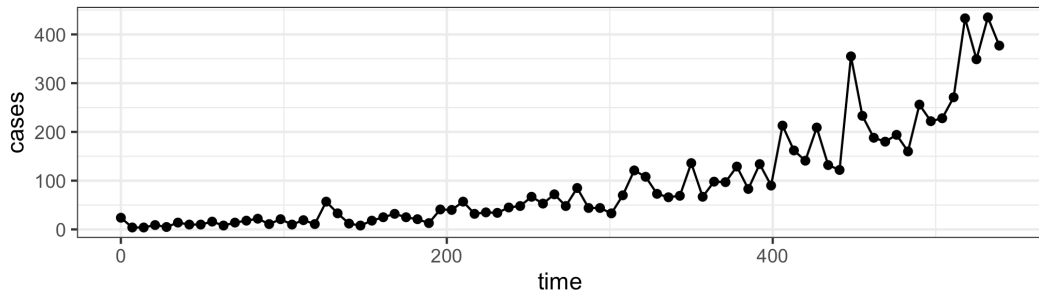
// 2. Draw simultaneously
// (num_paths, size, &rates, dt, &trans)
reulermultinom(2, E, &rates[0], dt, &trans[0]);

// 3. Extract results
double trans_I = trans[0];
double trans_A = trans[1];
```

Note: The `&` symbol just tells C exactly where your lists are stored in memory.

Introduction I

- ▶ Our goal: practice what we learned, design a model and make predictions.
- ▶ **Scenario:** A zombie-apocalyptic infection has spread on a small island nation.
- ▶ Simulated data available on course website: weekly data for 1.5 years



Goals

We want to practice a more realistic scenario: some data on an outbreak are observed, and we want to design and fit a model using known features about the outbreak.

- ▶ Design a model based on known facts.
- ▶ Implement a functional model in `pomp`.
- ▶ Get preliminary parameter estimates using `mif2`.
- ▶ Use fitted model to perform inference.

The Anomaly

- ▶ **The Setup:** Public health officials are tracking a bizarre neurological outbreak (colloquially “Zombies”).
- ▶ **The Hypothesis:** The pathogen circulated completely undetected for 2 months, causing no reportable symptoms. After 2 months, it mutated.
- ▶ **The Mutation:** A high (unknown) percentage of exposed individuals turn into aggressive Zombies; the remaining proportion develop a rash and become permanently immune.
- ▶ **Spread:** The virus spreads from contacts between current zombies + exposed individuals and susceptible survivors.
- ▶ **Containment:** Zombies do not die naturally. They are destroyed via mass-action combat driven by surviving humans (Susceptible + Immune).

The Fog of War (Measurement Model)

- ▶ We only partially observe total zombies, though reporting is high.
- ▶ The exposed class (those that may become zombies) is completely invisible.
- ▶ Reporting infrastructure decays (exponentially) as the total zombie horde (Z) grows.

Group Exercise: Model Conceptualization (20 mins)

Before touching R, draw the biological reality:

- ▶ **Compartments:** What compartments should we include based on what we know?
- ▶ **Transitions:** Draw a diagram that depicts a compartmental model to describe disease progression and population counts. Pay special attention to compartments that have multiple exit arrows!
- ▶ **Rates:** How should we define the transition rates? Should we include births and natural deaths to account for the long timescale of the data?

Translating Concepts to Code

Now, we translate the diagram into a continuous-time, stochastic model using `pomp`.

The Pieces of the Puzzle:

1. `rinit`: How should we initialize the model at $t = 0$?
2. `rprocess`: Implement the transition rates. How do we implement the mass-action combat destruction of zombies?
3. `rmeasure`: How do we link our latent state to the observed “zombie cases”?
4. **The Object**: Bind the data, times, and C-snippets together.

Implementation Hints

- ▶ **The Mutation Time:** The mutation doesn't unlock until 60 days post-infection. You can use a time condition in C: `double rate_E_out = (t < t_mut) ? 0.0 : gamma;`
- ▶ **Data Offset vs Initialization:** Our data starts exactly when the zombies appear (Week 0). However, Patient Zero was infected roughly 60 days *before* this. Set `t0 = -60` in your `pomp` object to allow the unobserved E compartment to grow before the data begins.
- ▶ **Incidence Tracking:** We only observe *new* cases. Make sure to track incident zombies in a separate variable and include it in `accumvars` in the `pomp` call.

Independent Exercise: Build the Model (45 mins)

Your Task: Work in your groups to translate your conceptual model into pomp C-snippets.

1. Write `rinit`, `rprocess`, `rmeasure`, and `dmeasure` snippets.
2. Build the `pomp` object.
3. Choose a reasonable set of starting parameters and use `simulate()` to ensure your model produces an outbreak curve that roughly matches the shape of the data.

Ionides, E. L., C. Breto, J. Park, R. A. Smith, and A. A. King. 2017. "Monte Carlo Profile Confidence Intervals for Dynamic Systems." *J R Soc Interface* 14 (July): 20170126. <https://doi.org/10.1098/rsif.2017.0126>.